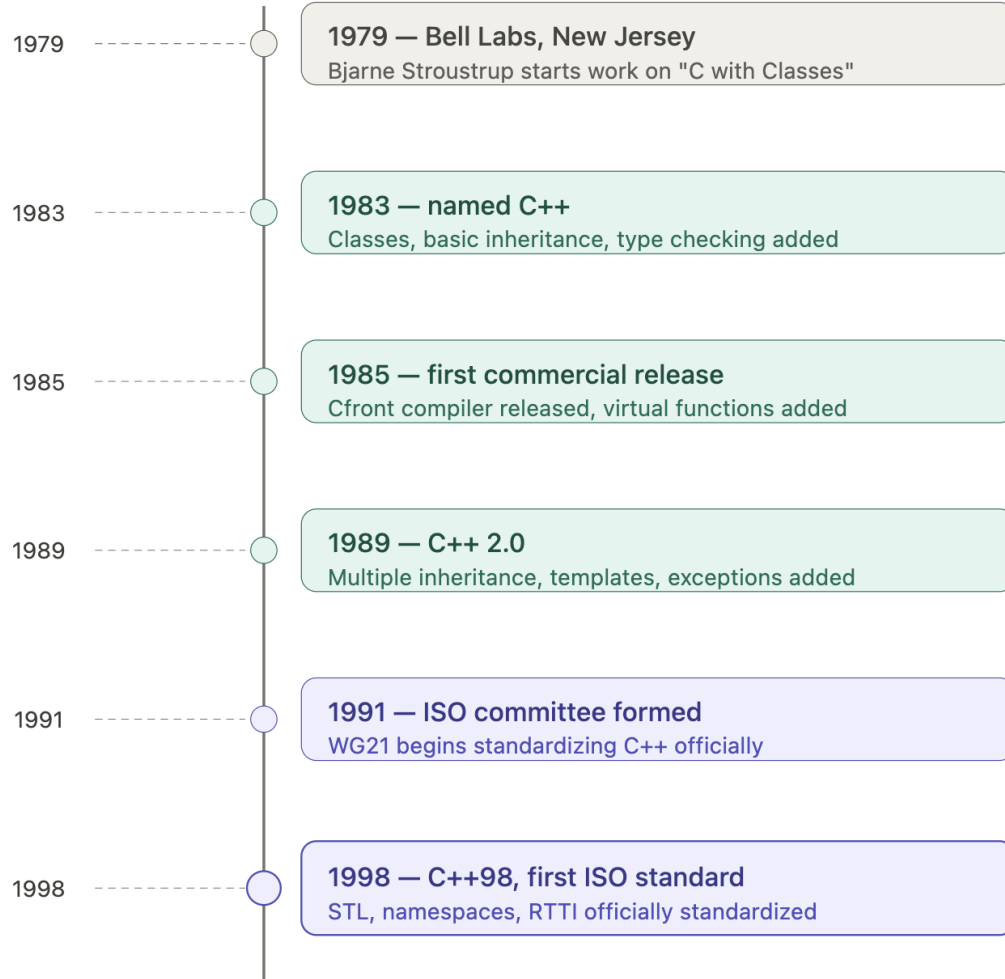


History of C++





C++ STANDARDS TIMELINE

Evolution of C++: Key Features, Limitations, and Years Introduced



1998

C++98



KEY NEW FEATURES

- First ISO standard
- STL (containers, iterators, algorithms)
- Templates
- Exceptions
- Namespaces

LIMITATIONS / GAPS

- No modern memory management
- No move semantics
- Verbose syntax
- Limited type inference

2003

C++03



KEY NEW FEATURES

- Bug fixes and clarifications over C++98
- Improved portability

LIMITATIONS / GAPS

- No major new features
- Still lacks modern programming constructs

2011

C++11



KEY NEW FEATURES

- Auto keyword
- Range-based for loop
- Lambda expressions
- Smart pointers (unique_ptr, shared_ptr)
- Move semantics
- nullptr
- Thread support
- constexpr
- Uniform initialization

LIMITATIONS / GAPS

- Increased complexity
- Slower compiler adoption initially
- Some features hard to master

2014

C++14



KEY NEW FEATURES

- Generic lambdas
- Return type deduction
- Improved constexpr
- Relaxed restrictions
- Minor STL improvements

LIMITATIONS / GAPS

- Mostly incremental
- No major paradigm shift

2017

C++17



KEY NEW FEATURES

- Structured bindings
- if constexpr
- Parallel STL
- std::optional, std::variant, std::any
- Filesystem library
- Fold expressions

LIMITATIONS / GAPS

- No reflection
- Modules not yet included
- Compile times still high

2020

C++20



KEY NEW FEATURES

- Concepts
- Ranges library
- Coroutines
- Modules
- Three-way comparison (<=>)
- consteval, constexpr
- Improved lambdas

LIMITATIONS / GAPS

- Compiler support initially incomplete
- Modules adoption slow
- Complexity increased further

2023

C++23



KEY NEW FEATURES

- std::expected
- std::print
- Deducing this
- Improved ranges
- Stacktrace
- mdspan
- More constexpr support

LIMITATIONS / GAPS

- Still evolving adoption
- Some features depend on library/compiler maturity

~2026
(EXPECTED)

C++26



KEY NEW FEATURES

- Reflection (planned)
- Pattern matching
- Executors
- Further ranges and concurrency improvements

LIMITATIONS / GAPS

- Not finalized
- Inconsistent support
- Subject to change

KEY TAKEAWAYS



C++11 is the biggest turning point – introduces Modern C++.



C++14 & C++17 refine usability – cleaner, safer, more expressive.



C++20 is revolutionary again – concepts + ranges change how templates are written.



C++23+ focus on polish, safety, and developer productivity.

LEGEND

- Released Standard
- Upcoming / In Progress
- Major Milestone



FOR TEACHING (UG LEVEL)

- ✓ Start with C++11 core features
- ✓ Introduce C++17 for productivity
- ✓ Selectively teach C++20 concepts/ranges (important for abstraction thinking)

C++ PROGRAMMING SERIES

GENERIC PROGRAMMING IN C++

Function Templates · Class Templates · Template Specialization · STL · Real-World Projects

Function Templates

Class Templates

Template Params

Specialization

STL & Generics

Real Projects

TABLE OF CONTENTS

01

Introduction to Generic Programming

Motivation, principle, real-world need

03

Class Templates

Generic containers, member functions

05

Template Specialization

Full & partial specialization, SFINAE

07

Real-World Projects

Stack, Pair, Matrix, Type-safe logger

02

Function Templates

Syntax, type deduction, overloading

04

Template Parameters

Non-type params, default params, variadic

06

STL & Generic Algorithms

vector, sort, algorithms - all use templates

08

Best Practices & Summary

Tips, pitfalls, modern C++ generics

SECTION 01

Introduction to Generic Programming

WHAT IS GENERIC PROGRAMMING?

Generic Programming is a programming paradigm in which algorithms and data structures are written in terms of **types to be specified later**, instantiated when needed for specific types provided as parameters. In C++, this is achieved through **templates**.

Write Once

Implement an algorithm once; use it with int, double, string, or any custom type without rewriting.

Zero Overhead

Templates are resolved at compile time – no runtime cost. As fast as hand-written type-specific code.

Type Safety

Errors are caught at compile time, not runtime. No unsafe casts or void* tricks needed.

THE PROBLEM WITHOUT TEMPLATES — CODE DUPLICATION

WITHOUT TEMPLATES (Bad)

```
// Without templates – must write for EVERY type!
int    addInt    (int a,    int b)    { return a + b; }
double addDouble(double a, double b) { return a + b; }
float  addFloat  (float a,  float b)  { return a + b; }
string addString(string a, string b) { return a + b; }
// ... and again for every type you need!

int    maxInt    (int a,    int b)    { return a>b?a:b; }
double maxDouble(double a, double b) { return a>b?a:b; }
// ...

// Even worse: void* approach (NOT type-safe!)
void* addVoid(void* a, void* b, int type) {
    if (type == 0) { // int
        int* ia=(int*)a;
        int* ib=(int*)b;
        int* r = new int(*ia + *ib);
        return r;      // ← dangerous! caller must cast
    }
    // else if (type == 1)... etc
}
```

Problems This Creates

✗ Code explosion

$N \text{ types} \times M \text{ functions} = N \times M \text{ implementations}$

✗ Maintenance hell

Fix a bug? Fix it in EVERY type-specific version

✗ Inconsistency

Easy for versions to drift out of sync

✗ void* is unsafe

No type checking – crash at runtime, not compile time

✗ Readability

Dozens of nearly-identical functions clutter the codebase

✓ **Solution: C++ Templates**
Write once, use with any type!

THE SOLUTION — C++ TEMPLATES AT A GLANCE

TEMPLATES — ONE FUNCTION, ANY TYPE

```
// — With templates: ONE implementation for ALL types —  
template <typename T>  
T add(T a, T b) { return a + b; }
```

```
// Compiler generates type-specific code on demand:  
int    r1 = add<int>(3, 4);           // T=int    → add(int,int)  
double r2 = add<double>(1.5, 2.5);    // T=double → add(double,double)  
string r3 = add<string>("Hi", "!");  // T=string → add(string,string)  
// Works with ANY type that supports operator+
```

```
// — Same with a max function —  
template <typename T>  
const T& maxVal(const T& a, const T& b) { return (a > b) ? a : b; }
```

```
cout << maxVal(7, 3);           // 7  
cout << maxVal(3.14, 2.71);     // 3.14  
cout << maxVal('z', 'a');       // z
```

```
// — Key insight —  
// Templates are resolved entirely at COMPILE TIME.  
// No performance penalty. Full type safety.
```

0

Runtime
Overhead

∞

Types
Supported

C++98

Templates
Introduced

Template instantiation = compiler generates type-specific code automatically

WHERE DO YOU SEE GENERIC PROGRAMMING EVERY DAY?

`std::vector<T>`

Dynamic array that holds ANY type — `vector<int>`, `vector<string>`, `vector<Employee>`.

`std::sort()`

Sorts ANY container of ANY comparable type — integers, strings, custom objects with `<` operator.

▫ `Smart Pointers`

`unique_ptr<T>`, `shared_ptr<T>` — generic memory management for any heap-allocated type.

▫ `std::map<K,V>`

Dictionary mapping ANY key type to ANY value — `map<string,int>`, `map<int,Employee>`.

`std::find()`

Searches ANY range for ANY element — works on arrays, vectors, lists, deques uniformly.

▫ `std::pair<A,B>`

Generic two-value container — `pair<string,int>` for name+age, `pair<int,double>` for coordinates.

SECTION 02

Function Templates

Syntax · Type deduction · Multiple type params · Overloading

FUNCTION TEMPLATE — SYNTAX & ANATOMY

SYNTAX ANATOMY

```
// — ANATOMY of a function template —————
template <typename T>           // ← template parameter list
//      ^^^^^^^ 'typename' and 'class' are equivalent here
T maximum(T a, T b) {           // ← T used as return type AND parameter type
    return (a > b) ? a : b;      // ← requires operator> to be defined for T
}

// — Using it (EXPLICIT instantiation) —————
int    i = maximum<int>(10, 20);    // explicitly: T=int
double d = maximum<double>(3.7, 1.2); // explicitly: T=double

// — Using it (IMPLICIT — compiler DEDUCES T) —————
auto r1 = maximum(10, 20);          // compiler infers T=int  (both args are int)
auto r2 = maximum(3.7, 1.2);        // compiler infers T=double
auto r3 = maximum('a', 'z');        // compiler infers T=char

// — 'class' keyword (identical to 'typename' for type params) ———
template <class U>                 // 'class' and 'typename' are interchangeable
U absolute(U val) {
    return (val < 0) ? -val : val;
}
```

⚡ template<typename T> must appear on the line BEFORE the function — it's NOT a separate declaration but part of the template definition.

MULTIPLE TEMPLATE TYPE PARAMETERS

MULTIPLE TYPE PARAMS

```
// — Two type parameters —————
template <typename T1, typename T2>
void printPair(T1 first, T2 second) {
    cout << "(" << first << ", " << second << ")" << endl;
}

printPair(42, "hello");           // T1=int, T2=const char*
printPair(3.14, true);            // T1=double, T2=bool
printPair("key", 100);            // T1=const char*, T2=int

// — Mixed types with conversion —————
template <typename RetType, typename T1, typename T2>
RetType addAs(T1 a, T2 b) {
    return static_cast<RetType>(a) + static_cast<RetType>(b);
}

double result = addAs<double>(5, 2.7); // int + double → double
// RetType must be explicit; T1,T2 are deduced from arguments

// — Real-world example: type-safe map insert —————
template <typename K, typename V>
bool insertIfAbsent(map<K,V>& m, const K& key, const V& val) {
    if (m.find(key) == m.end()) {
        m[key] = val;
        return true; // inserted
    }
    return false; // already present
}
```

TEMPLATE TYPE DEDUCTION — HOW THE COMPILER FIGURES OUT T

TYPE DEDUCTION

// — Auto deduction: compiler deduces T from arguments —————

```
template <typename T>
T square(T x) { return x * x; }
```

```
square(4);           // T deduced as int    → returns int
square(2.5);         // T deduced as double → returns double
square(3.0f);        // T deduced as float  → returns float
```

// — Deduction with references —————

```
template <typename T>
void inspect(const T& val) {
    cout << "Type size: " << sizeof(T) << " bytes\n";
}

inspect(42);          // T=int, const int&
inspect(3.14);        // T=double, const double&
inspect("hello");     // T=char[6], const char(&)[6] ← interesting!
```

// — When deduction FAILS (must specify explicitly) —————

```
template <typename T>
T defaultValue() { return T{}; } // T appears only in return type
```

```
// defaultValue();           // ✗ ERROR: cannot deduce T
int v = defaultValue<int>(); // ✔ must explicitly provide T=int
```

// — C++17 Class Template Argument Deduction (CTAD) —————

```
// std::pair<int,double> p = {1, 3.14}; ← old way
auto p = std::pair(1, 3.14);           // ← C++17 CTAD deduces <int,double>
```

FUNCTION TEMPLATE OVERLOADING

OVERLOADING TEMPLATES

```
// — Template + non-template overloads —————
template <typename T>
T maxVal(T a, T b) { // generic template
    cout << "[template] ";
    return (a > b) ? a : b;
}

// Non-template overload for const char*
const char* maxVal(const char* a, const char* b) {
    cout << "[non-template] ";
    return (strcmp(a, b) > 0) ? a : b; // compare strings correctly!
}

// Template overload for pointer types
template <typename T>
T* maxVal(T* a, T* b) {
    cout << "[pointer template] ";
    return (*a > *b) ? a : b;
}

int main() {
    cout << maxVal(3, 5) << "\n"; // uses generic template (T=int)
    cout << maxVal(3.14, 2.72) << "\n"; // uses generic template (T=double)
    cout << maxVal("apple", "mango") << "\n"; // uses const char* non-template!
    int x=10, y=20;
    cout << *maxVal(&x,&y) << "\n"; // uses pointer template
}
```

// Overload Resolution Priority:

Overload Resolution Priority

1

Exact non-template match

2

Template specialization match

3

Template with type deduction

4

Implicit conversion (non-template)

PRACTICAL FUNCTION TEMPLATES — REAL EXAMPLE

PRACTICAL EXAMPLES

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// — Generic swap —
template <typename T>
void genericSwap(T& a, T& b) {
    T temp = a; a = b; b = temp;
}

// — Generic print array / vector —
template <typename T>
void printAll(const vector<T>& v, const string& sep = ", ") {
    for (size_t i=0; i<v.size(); ++i) {
        cout << v[i];
        if (i+1 < v.size()) cout << sep;
    }
    cout << "\n";
}

// — Generic findMax in array —
template <typename T>
T findMax(const T arr[], int size) {
    T best = arr[0];
    for (int i=1; i<size; ++i)
        if (arr[i] > best) best = arr[i];
    return best;
}
```

SECTION 03

Class Templates

Generic containers · Member functions · Nested types · Inheritance

CLASS TEMPLATE — SYNTAX & STRUCTURE

CLASS TEMPLATE SYNTAX

```
// — A generic Box class —————
template <typename T>           // ← template parameter
class Box {
private:
    T value_;                  // ← T used as member type
    string label_;

public:
    // Constructor
    Box(T val, const string& lbl="") : value_(val), label_(lbl) {}

    // Member functions – T used in parameters/return types
    T    getValue() const { return value_; }
    void setValue(T val) { value_ = val; }
    string getLabel() const { return label_; }

    void display() const {
        cout << "Box[" << label_ << "] = " << value_ << "\n";
    }
    // Operator overloading works too
    bool operator<(const Box<T>& other) const {
        return value_ < other.value_;
    }
};

// — Using the class template —————
Box<int>    intBox(42, "age");
Box<double> dblBox(3.14, "pi");
Box<string> strBox("Hello", "greeting");

intBox.display();    // Box[age] = 42
dblBox.display();    // Box[pi] = 3.14
strBox.display();    // Box[greeting] = Hello
```

DEFINING MEMBER FUNCTIONS OUTSIDE THE CLASS

MEMBER FUNCTIONS OUTSIDE

// — Declaration inside class —————

```
template <typename T>
class Stack {
    vector<T> data_;
public:
    void push(const T& item); // ← just declared
    void pop();
    T top() const;
    bool empty() const;
    int size() const;
    void print() const;
};
```

// —Definition OUTSIDE class — must repeat template header—

```
template <typename T> // ← required every time
void Stack<T>::push(const T& item){ //← Stack<T>, not just Stack
    data_.push_back(item);
}
```

```
template <typename T>
void Stack<T>::pop() {
    if (data_.empty()) throw underflow_error("Stack is empty");
    data_.pop_back();
}
```

```
template <typename T>
T Stack<T>::top() const {
    if (data_.empty()) throw underflow_error("Stack is empty");
    return data_.back();
}
```

```
template <typename T>
void Stack<T>::print() const {
    cout << "Stack (top→bottom): ";
    for (int i = data_.size()-1; i >= 0; --i)
        cout << data_[i] << " ";
    cout << "\n";
}
```

REAL PROJECT: GENERIC STACK — FULL IMPLEMENTATION

GENERIC STACK PROJECT

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

template <typename T>
class Stack {
    vector<T> data_;
    int capacity_;
public:
    Stack(int cap=100) : capacity_(cap) {}

    void push(const T& item) {
        if ((int)data_.size() >= capacity_)
            throw overflow_error("Stack overflow");
        data_.push_back(item);
    }

    void pop() {
        if(empty())
            throw underflow_error("Stack underflow");
        data_.pop_back(); }

    T top() const {
        if(empty())
            throw underflow_error("Stack empty");
        return data_.back(); }

    bool empty() const { return data_.empty(); }

    int size() const { return data_.size(); }

    void print() const {
        cout << "[top] ";
        for (int i=data_.size()-1; i>=0; --i)
            cout << data_[i] << " ";
        cout << "[bottom]\n";
    }
};

int main() {
    Stack<int> intStack;
    intStack.push(10); intStack.push(20); intStack.push(30);
    intStack.print();           // [top] 30 20 10 [bottom]
    cout << intStack.top();      // 30
    intStack.pop();
    intStack.print();           // [top] 20 10 [bottom]

    Stack<string> strStack;
    strStack.push("Pakistan"); strStack.push("China");
    strStack.push("USA");
    strStack.print();           // [top] USA China Pakistan [bottom]
    cout << "Same Stack class. Different type!\n";
}
```

REAL PROJECT: GENERIC PAIR

GENERIC PAIR PROJECT

```
// — Two-type generic Pair —
template <typename T1, typename T2>
class Pair {
public:
    T1 first;
    T2 second;

    Pair(T1 f, T2 s) : first(f), second(s) {}

    void display() const {
        cout << "(" << first << ", " << second << ")\n";
    }

    // Swap (only if T1==T2)
    template <typename U = T1>
    void swapValues(T1& a, T2& b) { swap(a, b); }

    bool operator==(const Pair<T1,T2>& o) const {
        return first == o.first && second == o.second;
    }
};

// — Usage —
int main() {
    Pair<string, int> student("Ali", 20);
    Pair<double, double> coord(40.7128, -74.0060); // NYC lat/lon
```

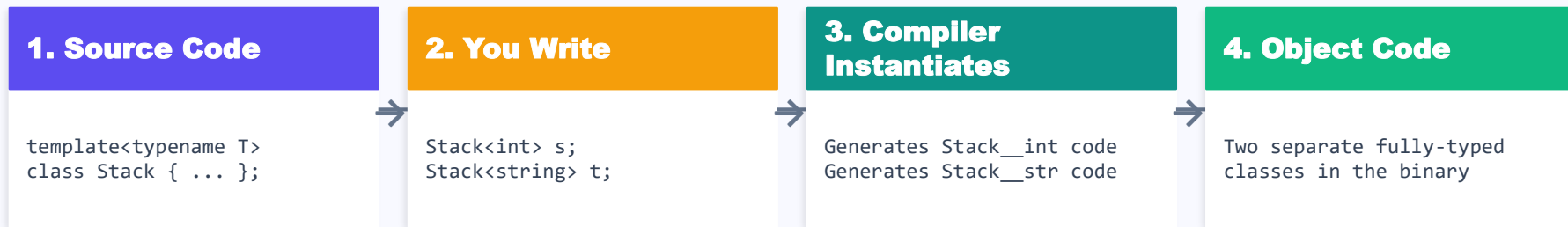
```
    Pair<string, string> country("Pakistan","Islamabad");

    student.display(); // (Ali, 20)
    coord.display();   // (40.7128, -74.006)
    country.display();  // (Pakistan, Islamabad)

    // Array of Pairs
    vector<Pair<string,int>> scores =
        {"Alice",95},{"Bob",87},{"Carol",92}};

    for (auto& p : scores) p.display();
}
```

CLASS TEMPLATE INSTANTIATION — HOW THE COMPILER WORKS



✂ **KEY:** Templates are NOT code — they are BLUEPRINTS. Code only gets generated when you instantiate (use) a specific type.

INSTANTIATION RULES

```
// — Implicit instantiation —————  
Stack<int>    s1;    // Compiler generates Stack<int> class for you (implicit)  
Stack<string> s2;    // Compiler generates Stack<string> class for you (implicit)  
  
// — Explicit instantiation (forces code generation in this TU) ————  
template class Stack<double>;    // Explicit — code generated immediately here  
  
// — Important rule: template definition must be visible —————  
// Put template definitions in .h files (not .cpp), OR use:  
//   → "inclusion model": #include the .cpp in the .h  
//   → explicit instantiation declarations in .cpp files  
// Reason: compiler needs full template body to generate code at each use site.
```

SECTION 04

Template Parameters

Non-type params · Default params · Variadic templates

NON-TYPE TEMPLATE PARAMETERS

NON-TYPE PARAMETERS

```
// — Templates can take VALUES, not just types —————  
template <typename T, int SIZE> // SIZE is a non-type parameter  
                                //(compile-time constant)
```

```
class FixedArray {  
    T data_[SIZE];    // SIZE used to declare fixed-size array  
    int count_ = 0;  
public:  
    void add(const T& val) {  
        if (count_ >= SIZE) throw overflow_error("Array full");  
        data_[count_++] = val;  
    }  
    T& operator[](int i) { return data_[i]; }  
    int capacity() const { return SIZE; }  
    int size() const { return count_; }  
};
```

```
// — Each SIZE creates a DIFFERENT type —————  
FixedArray<int,10>    scores;    // 10-element int array  
FixedArray<double,5>  readings;   // 5-element double array  
FixedArray<string,3>  names;     // 3-element string array
```

```
// FixedArray<int,10> and FixedArray<int,20>  
// are DIFFERENT types!
```

```
// — std::array uses the same pattern (C++11 standard library)—
```

```
#include <array>  
std::array<int, 5>    a = {1,2,3,4,5};    // non-type param: 5  
std::array<double, 10> b;                // non-type param: 10
```

```
// — Other valid non-type parameter kinds —————
```

```
template <int N>  
void printN() {  
    cout << N; } // integer
```

```
template <bool Debug>  
void log(string m){  
    if constexpr(Debug)  
        cout<<m; } // bool
```

```
template <char Sep>  
void printSep() {  
    cout << Sep; } // char
```

DEFAULT TEMPLATE PARAMETERS

DEFAULT TEMPLATE PARAMS

```
// — Type parameters can have defaults —
template <typename T = int, typename Allocator =
std::allocator<T>>
class MyVector {
    vector<T, Allocator> data_;
public:
    void push(const T& v) { data_.push_back(v); }
    T    at(int i) const { return data_.at(i); }
    int  size() const    { return data_.size(); }
};

MyVector<> intVec;    // T=int (default), uses std::allocator
MyVector<double> dblVec; //T=double, uses std::allocator<double>
MyVector<string> strVec; // T=string, uses std::allocator<string>
```

```
// — Non-type defaults —
template <typename T, int N = 10, bool Checked = false>
class SmartArray {
    T data_[N];
public:
    T& at(int i) {
        if constexpr (Checked) { // compile-time if (C++17)
            if (i < 0 || i >= N)
                throw out_of_range("index out of bounds");
        }
        return data_[i];
    }

    int capacity() { return N; }
};

SmartArray<int> a1; // T=int, N=10, Checked=false
SmartArray<double, 5> a2; // T=double, N=5, Checked=false
SmartArray<string, 3, true> a3; // T=string, N=3, Checked=true
// (bounds checking on!)
```

VARIADIC TEMPLATES — ARBITRARY NUMBER OF PARAMETERS (C++11)

VARIADIC TEMPLATES

```
// — Variadic function template —————
template <typename T>
void print(T val) {                // Base case (recursion stop)
    cout << val << "\n";
}
template <typename T, typename... Rest> // ... = parameter pack
void print(T first, Rest... rest) {    // first + rest...
    cout << first << " ";
    print(rest...);                  // Recursive call with remaining args
}
print(1, 2.5, "hello", 'c', true);
// Output: 1 2.5 hello c 1
```

```
// — Variadic template: sum of any number of values ———
template <typename T>
T sum(T val) { return val; }        // Base case

template <typename T, typename... Rest>
T sum(T first, Rest... rest) {
    return first + sum(rest...);
}
cout << sum(1, 2, 3, 4, 5)    << "\n"; // 15
cout << sum(1.1, 2.2, 3.3)    << "\n"; // 6.6
```

```
#include <iostream>
using namespace std;

template <typename... Args>
void print(Args... args) {
    (cout << ... << args) << "\n"; //fold expression(C++17)
}

int main() {
    print(1, 2, 3);                // 123
    print("hello", " ", 42);       // hello 42
    print(3.14, '!', true);        // 3.14!1
}
```

SECTION 05

Template Specialization

Full specialization · Partial specialization · SFINAE · Concepts

FULL TEMPLATE SPECIALIZATION

FULL SPECIALIZATION

// — Generic template (primary)

```
template <typename T>
class TypeInfo {
public:
    static string name()      { return "unknown"; }
    static bool isNumeric()   { return false; }
    static T    defaultValue() { return T{}; }
};
```

// — Full specialization for int

```
template <>      // ← empty parameter list = full specialization
class TypeInfo<int> {    // ← concrete type in angle brackets
public:
    static string name()      { return "int"; }
    static bool isNumeric()   { return true; }
    static int  defaultValue() { return 0; }
};
```

// — Full specialization for string

```
template <>
class TypeInfo<string> {
public:
    static string name()      { return "string"; }
```

```
    static bool isNumeric()   { return false; }
    static string defaultValue(){ return ""; }
};
```

// — Usage

```
cout << TypeInfo<int>::name()      << "\n"; // int
cout << TypeInfo<double>::name()   << "\n"; // unknown (uses
generic)
cout << TypeInfo<string>::name()   << "\n"; // string
cout << TypeInfo<int>::isNumeric() << "\n"; // 1 (true)
```

PARTIAL TEMPLATE SPECIALIZATION

PARTIAL SPECIALIZATION

```
// — Primary template (generic) —————
template <typename T1, typename T2>
class SmartPair {
public:
    T1 first; T2 second;
    SmartPair(T1 f, T2 s): first(f), second(s) {}
    void display() { cout << "Generic: (" << first << ", " <<
second << ")\n"; }
};
```

```
// — Partial specialization: both types the same —————
template <typename T>
class SmartPair<T, T> {          // specialization for T1==T2
public:
    T first, second;
    SmartPair(T f, T s): first(f), second(s) {}
    T sum() const { return first + second; } // bonus: sum
available!
    void display() { cout << "Same-type: (" << first << ", " <<
second << ")\n"; }
};
```

```
// — Partial specialization: second type is pointer —————
template <typename T1, typename T2>
class SmartPair<T1, T2*> {      // specialization when T2 is
a pointer
```

```
public:
    T1 first;
    T2* second;
    SmartPair(T1 f, T2* s): first(f), second(s) {}
    void display() { cout << "Pointer: (" << first << ", " <<
*second << ")\n"; }
};

int main() {
    SmartPair<int,string> p1(1,"hello"); // uses primary
    SmartPair<int,int> p2(3,7); //uses same-type partial spec
    cout << p2.sum() << "\n";           // 10 – bonus method!
    int x=42;
    SmartPair<string,int> p3("val",&x); // uses pointer
partial spec
    p1.display(); p2.display(); p3.display();
}
```

FULL SPECIALIZATION: FUNCTION TEMPLATE — `const char*` Case

FUNCTION SPECIALIZATION

// — Generic function template

```
template <typename T>
bool isEqual(T a, T b) {
    return a == b;           // generic equality
}
```

// — Full specialization for `const char*` (needs `strcmp`) —

```
template <>
bool isEqual<const char*>(const char* a, const char* b) {
    return strcmp(a, b) == 0; // pointer comparison vs content
    comparison
}
```

```
int main() {
    cout << isEqual(5, 5) << "\n"; // 1 (true) - generic
    cout << isEqual(3.14, 3.14) << "\n"; // 1 (true) - generic
    cout << isEqual("hello", "hello") << "\n"; // 1 (true) - specialized!
    // Without specialization, comparing "hello"=="hello"
    compares POINTERS
    // (two different string literals → may be false even if
    content matches)
}
```

// — Printing template (generic vs specialized)

```
template <typename T>
void prettyPrint(T val) {
    cout << val << "\n";
}
```

```
template <>
void prettyPrint<bool>(bool val) { // specialized for bool
    cout << (val ? "true" : "false") << "\n"; // nicer than 0/1
}
```

```
template <>
void prettyPrint<double>(double val) { // specialized for double
    cout << fixed << setprecision(4) << val << "\n";
}
```

```
// prettyPrint(42)           → 42
// prettyPrint(true)         → true      (not 1!)
// prettyPrint(3.14159)      → 3.1416    (formatted!)
```

SFINAE & TYPE TRAITS — COMPILE-TIME TYPE INSPECTION

SFINAE & TYPE_TRAITS

```
// — SFINAE = Substitution Failure Is Not An Error —————  
// When template substitution fails, that overload is silently removed  
// (not a compile error). Other overloads may still match.  
  
#include <type_traits>  
  
// Enable this function ONLY for arithmetic types (int, double, float...)  
template <typename T>  
typename enable_if<is_arithmetic<T>::value, T>::type //← SFINAE condition  
safeDivide(T a, T b) {  
    if (b == T(0)) throw invalid_argument("Division by zero");  
    return a / b;  
}  
  
// — C++17: if constexpr – cleaner alternative —————  
template <typename T>  
void describe(T val) {  
    if constexpr (is_integral<T>::value)  
        cout << val << " is an integer\n";  
    else if constexpr (is_floating_point<T>::value)  
        cout << val << " is a float\n";  
    else  
        cout << val << " is some other type\n";  
}
```

SECTION 06

STL & Generic Algorithms

vector · sort · find · iterator pattern · all use templates

THE STL IS BUILT ENTIRELY ON TEMPLATES

Containers

`vector<T>`

`list<T>`

`deque<T>`

`set<T>`

`multiset<T>`

`map<K,V>`

`multimap<K,V>`

`unordered_map<K,V>`

`stack<T>`

`queue<T>`

`priority_queue<T>`

`array<T,N>`

`forward_list<T>`

Algorithms

`sort(first,last)`

`find(first,last,val)`

`find_if(f,l,pred)`

`count(f,l,val)`

`copy(f,l,dest)`

`transform(f,l,d,fn)`

`accumulate(f,l,init)`

`max_element(f,l)`

`binary_search(f,l,v)`

`lower_bound(f,l,v)`

`upper_bound(f,l,v)`

`reverse(f,l)`

`unique(f,l)`

Iterators

`begin() / end()`

`rbegin() / rend()`

`cbegin() / cend()`

`InputIterator`

`OutputIterator`

`ForwardIterator`

`BidirectionalIter`

`RandomAccessIter`

`istream_iterator<T>`

`ostream_iterator<T>`

`back_inserter`

`front_inserter`

`inserter`

std::vector<T> — THE MOST-USED GENERIC CONTAINER

```
std::vector<T>
```

```
#include <vector>
#include <algorithm>
using namespace std;

// — vector is a class template: template<class T, class
Alloc=...> —
vector<int>    nums    = {5,3,8,1,9,2,7,4,6};
vector<string> names   = {"Alice","Bob","Carol","Dave"};
vector<double> temps;

// — Common operations —————
nums.push_back(10);    // add at back      O(1) amortized
nums.pop_back();       // remove from back O(1)
nums.insert(nums.begin()+2, 99); // insert at position O(n)
nums.erase(nums.begin()); // erase first element O(n)
int n = nums[3];        // element access O(1) unchecked
int m = nums.at(3);     // element access O(1) checked (throws)
int sz = nums.size();   // current size
nums.reserve(100);      // pre-allocate capacity (avoids
realloc)

// — Sorting (works because < is defined for int) ———
sort(nums.begin(), nums.end());    // ascending
sort(nums.rbegin(), nums.rend());  // descending
```

```
// — Sort custom objects with comparator —————
struct Student { string name; int grade; };
vector<Student> students =
{{"Alice",92},{ "Bob",87},{ "Carol",95}};
sort(students.begin(), students.end(),
    [](const Student& a, const Student& b)
    {return a.grade > b.grade; });
// Carol(95), Alice(92), Bob(87)
```

```
// — Range-based for (works with any container that has
begin/end) ———
for (const auto& s : students)
    cout << s.name << ": " << s.grade << "\n";
```

STL GENERIC ALGORITHMS — WITH LAMBDA

GENERIC ALGORITHMS

```
#include <algorithm>
#include <vector>
#include <numeric>
using namespace std;

vector<int> nums = {1,2,3,4,5,6,7,8,9,10};

// — find_if: find first even number
auto it = find_if(nums.begin(), nums.end(), [](int x){ return
x%2==0; });
if (it != nums.end()) cout << "First even: " << *it << "\n"; // // — remove_if + erase (erase-remove idiom)
2

// — count_if: count elements > 5
int big = count_if(nums.begin(), nums.end(), [](int x){ return
x>5; });
cout << "Count > 5: " << big << "\n"; // 5

// — transform: square all elements
vector<int> squares(nums.size());
transform(nums.begin(), nums.end(), squares.begin(), [](int x){
return x*x; });
// squares = {1,4,9,16,25,36,49,64,81,100}

// — accumulate: sum, product, string concat
int total = accumulate(nums.begin(), nums.end(), 0);
// 55
int product = accumulate(nums.begin(), nums.end(), 1,
multiplies<int>());
string joined= accumulate(names.begin(), names.end(),
string(""),
[](string acc, const string& s){ return
acc+s","; });

nums.erase(
remove_if(nums.begin(), nums.end(), [](int x){ return
x%2==0; }),
nums.end()); // Remove all even numbers: {1,3,5,7,9}

// — all_of / any_of / none_of
bool allPos = all_of (nums.begin(),nums.end(),[](int x){return
x>0;}); // true
bool anyNeg = any_of (nums.begin(),nums.end(),[](int x){return
x<0;}); // false
bool noneZero=none_of(nums.begin(),nums.end(),[](int x){return
x==0;}); // true
```

std::map<K,V> & std::unordered_map<K,V>

map & unordered_map

```
#include <map>
#include <unordered_map>
using namespace std;

// — map<K,V> – sorted, O(log n) operations


---


map<string,int> wordCount;
wordCount["apple"]++; wordCount["banana"]++;
wordCount["apple"]++; wordCount["cherry"]++;
// wordCount = { apple:2, banana:1, cherry:1 } (sorted by key!)

for (const auto& [word, count] : wordCount)           // C++17
    structured bindings
        cout << word << ": " << count << "\n";

// — unordered_map<K,V> – hash-based, O(1) average


---


unordered_map<string,int> phoneBook;
phoneBook["Alice"] = 555001;
phoneBook["Bob"] = 555002;
phoneBook["Carol"] = 555003;

// Safe lookup with find()
if (auto it = phoneBook.find("Bob"); it != phoneBook.end())
    cout << "Bob's number: " << it->second << "\n"; // 555002

// — Real use case: frequency counter


---


template <typename T>
map<T,int> frequency(const vector<T>& data) {
    map<T,int> freq;
    for (const auto& item : data) freq[item]++;
    return freq;
}

vector<string>
fruits={"apple","banana","apple","cherry","banana","apple"};
auto freq = frequency(fruits); // apple:3, banana:2, cherry:1

// — map vs unordered_map


---


// map:                sorted keys, O(log n), needs operator<
// unordered_map:      unsorted,    O(1) avg, needs hash + operator==
```

WRITING YOUR OWN ITERATOR-COMPATIBLE CONTAINER

CUSTOM ITERATOR CONTAINER

```
// A custom generic range/span that works with STL algorithms
template <typename T>
class SimpleRange {
    T* begin_;
    T* end_;
public:
    SimpleRange(T* b, T* e) : begin_(b), end_(e) {}
    SimpleRange(T* data, int size) : begin_(data),
                                     end_(data+size) {}

    T* begin() { return begin_; } // ← these make it work with
range-for
    T* end()   { return end_; }   //   and with std::
algorithms!
    const T* begin() const { return begin_; }
    const T* end()   const { return end_; }
    int size() const { return end_ - begin_; }
};

int main() {
    int arr[] = {5,3,8,1,9,2,7,4,6};
    SimpleRange<int> r(arr, 9);

    // Works with range-based for (because has begin/end)
    for (int x : r) cout << x << " ";

    // Works with STL algorithms (because iterators are
pointers)
    sort(r.begin(), r.end());
    int total = accumulate(r.begin(), r.end(), 0); // 45
    auto mx   = max_element(r.begin(), r.end()); // 9
    cout << "Sum=" << total << " Max=" << *mx << "\n";

    // Works with count_if, find_if, etc.
    int evens = count_if(r.begin(), r.end(), [](int x){ return
x%2==0; });
    cout << "Even count: " << evens << "\n";
}
```

SECTION 07

Real-World Projects

Generic Matrix · Type-safe Logger · Priority Queue · Registry

PROJECT 1: GENERIC MATRIX CLASS

GENERIC MATRIX

```
#include <vector>
#include <stdexcept>
using namespace std;

template <typename T>
class Matrix {
    int rows_, cols_;
    vector<vector<T>> data_;
public:
    Matrix(int r, int c, T init=T{})
        : rows_(r), cols_(c), data_(r, vector<T>(c, init)) {}

    T& at(int r, int c) { return data_.at(r).at(c); }
    const T& at(int r, int c) const { return data_.at(r).at(c); }

    int rows() const { return rows_; }
    int cols() const { return cols_; }

    Matrix<T> operator+(const Matrix<T>& o) const {
        if (rows_!=o.rows_||cols_!=o.cols_) throw
            invalid_argument("Size mismatch");
        Matrix<T> result(rows_,cols_);
        for (int r=0;r<rows_;++r)
            for (int c=0;c<cols_;++c)
                result.at(r,c) = data_[r][c] + o.data_[r][c];
        return result;
    }

    Matrix<T> operator*(const Matrix<T>& o) const {
        if (cols_!=o.rows_) throw invalid_argument("Incompatible
            for multiply");
        Matrix<T> result(rows_,o.cols_);
        for(int r=0;r<rows_;++r)
            for(int c=0;c<o.cols_;++c)
                for(int k=0;k<cols_;++k)
                    result.at(r,c) += data_[r][k] *
                        o.data_[k][c];
        return result;
    }

    void print() const {
        for (auto& row : data_) {
            for (auto& v : row) cout << v << " ";
            cout << "\n";
        }
    }
};

int main() {
    Matrix<int> A(2,2), B(2,2);
    A.at(0,0)=1; A.at(0,1)=2; A.at(1,0)=3; A.at(1,1)=4;
    B.at(0,0)=5; B.at(0,1)=6; B.at(1,0)=7; B.at(1,1)=8;
    auto C = A * B; C.print(); // [[19,22],[43,50]]

    Matrix<double> D(3,3,1.5); // 3x3 matrix of doubles
    cout << "Works for doubles too!\n";
}
```

PROJECT 2: GENERIC PRIORITY QUEUE

GENERIC PRIORITY QUEUE

```
#include <vector>
#include <functional>
#include <stdexcept>
using namespace std;

template <typename T, typename Compare = less<T>> // default: max-heap
class PriorityQueue {
    vector<T> heap_;
    Compare comp_; // comparator (type parameter!)

    void siftUp(int i) {
        while (i > 0) {
            int parent = (i-1)/2;
            if (comp_(heap_[parent], heap_[i])) {
                swap(heap_[parent], heap_[i]); i=parent; }
            else break;
        }
    }

    void siftDown(int i) {
        int n=heap_.size();
        while (true) {
            int best=i, l=2*i+1, r=2*i+2;
            if (l<n && comp_(heap_[best], heap_[l])) best=l;
            if (r<n && comp_(heap_[best], heap_[r])) best=r;
            if (best==i) break;
            swap(heap_[i], heap_[best]); i=best;
        }
    }

public:
    void push(const T& val) { heap_.push_back(val); siftUp(heap_.size()-1); }

    void pop() { if(empty()) throw underflow_error("empty");
        heap_[0]=heap_.back(); heap_.pop_back(); siftDown(0); }
    const T& top() const { if(empty()) throw underflow_error("empty");
        return heap_[0]; }
    bool empty() const { return heap_.empty(); }
    int size() const { return heap_.size(); }
};

int main() {
    PriorityQueue<int> maxPQ; // max-heap (default)
    PriorityQueue<int, greater<int>> minPQ; // min-heap

    for (int x : {5,1,9,3,7}) { maxPQ.push(x); minPQ.push(x); }
    cout << maxPQ.top() << "\n"; // 9 (largest)
    cout << minPQ.top() << "\n"; // 1 (smallest)

    // Use with custom type
    struct Task { string name; int priority; bool operator<(const Task&
o) const{return priority<o.priority;} };
    PriorityQueue<Task> taskQ;
    taskQ.push({"Send email", 2}); taskQ.push({"Fix crash", 10});
    taskQ.push({"Code review",5});
    cout << taskQ.top().name << "\n"; // Fix crash (highest priority)
}
```

PROJECT 3: TYPE-SAFE VARIADIC LOGGER

TYPE-SAFE VARIADIC LOGGER

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <ctime>
using namespace std;

enum class LogLevel { DEBUG, INFO, WARNING, ERROR };

class Logger {
    ofstream file_;
    LogLevel minLevel_;

    string levelStr(LogLevel lv) {
        switch(lv){case LogLevel::DEBUG:return "DEBUG";case
LogLevel::INFO:return "INFO";
                case LogLevel::WARNING:return "WARN";default:return
"ERROR";}
    }
    string timestamp() {
        time_t now=time(nullptr);
        char buf[20];
        strftime(buf,sizeof(buf),"%H:%M:%S",localtime(&now));
        return buf;
    }

    // — Variadic template: format any number of values into one
    message
    template <typename T>
    string toStr(const T& val) { return to_string(val); }
    string toStr(const string& s) { return s; }
    string toStr(const char* s) { return s; }
```

```
template <typename... Args>
string build(Args... args) {
    string result;
    // Fold expression (C++17): apply toStr to each arg
    ((result += toStr(args) + " "), ...);
    return result;
}

public:
    Logger(const string& filename, LogLevel min=LogLevel::DEBUG)
        : file_(filename, ios::app), minLevel_(min) {}

    template <typename... Args>
    void log(LogLevel level, Args&&... args) {
        if (level < minLevel_) return;
        string msg = "[" + timestamp() + "] [" + levelStr(level) + "] " +
build(args...);
        cout    << msg << "\n";
        file_   << msg << "\n";
    }
};

int main() {
    Logger logger("app.log", LogLevel::INFO);

    logger.log(LogLevel::INFO,    "Server started on port", 8080);
    logger.log(LogLevel::WARNING, "Memory usage:", 87, "% – high!");
    logger.log(LogLevel::ERROR,   "Connection failed to", "db.host.com",
"code:", 503);
    logger.log(LogLevel::DEBUG,   "This is suppressed (below INFO)");
}
```

PROJECT 4: GENERIC TYPE-SAFE REGISTRY

GENERIC FACTORY REGISTRY

```
#include <map>
#include <functional>
#include <memory>
using namespace std;

// — Generic factory/registry pattern ————— };
template <typename Base, typename Key = string>
class Registry {
    map<Key, function<unique_ptr<Base>()>> creators_;
public:
    // Register a creator for a key
    template <typename Derived>
    void reg(const Key& name) {
        creators_[name] = []{ return make_unique<Derived>(); };
    }

    // Register with a custom factory function
    void reg(const Key& name, function<unique_ptr<Base>()> fn) {
        creators_[name] = fn;
    }

    // Create an instance by name
    unique_ptr<Base> create(const Key& name) const {
        auto it = creators_.find(name);
        if (it == creators_.end())
            throw invalid_argument("Unknown type: " + name);
        return it->second();
    }

    vector<Key> keys() const {
        vector<Key> ks;
        for (auto& [k, _] : creators_) ks.push_back(k);
        return ks;
    }
};

// — Usage with a Shape hierarchy —————
struct Shape { virtual void draw()=0; virtual ~Shape()=default; };
struct Circle : Shape { void draw(){ cout<<"Drawing Circle\n"; } };
struct Rectangle : Shape { void draw(){ cout<<"Drawing Rectangle\n"; } };
struct Triangle : Shape { void draw(){ cout<<"Drawing Triangle\n"; } };

int main() {
    Registry<Shape> factory;
    factory.reg<Circle> ("circle");
    factory.reg<Rectangle>("rectangle");
    factory.reg<Triangle> ("triangle");

    vector<string> types = {"circle","rectangle","triangle","circle"};
    for (auto& t : types) {
        auto shape = factory.create(t); // returns unique_ptr<Shape>
        shape->draw();
    }
    // Available: circle, rectangle, triangle
}
```

PROJECT 5: GENERIC OBSERVER / EVENT SYSTEM

GENERIC OBSERVER PATTERN

```
#include <vector>
#include <functional>
using namespace std;

// — Generic observable that fires callbacks of type void(EventData)
template <typename EventData>
class Observable {
    vector<function<void(const EventData&>>> listeners_;
public:
    // Subscribe a listener (any callable: lambda, function ptr, method)
    void subscribe(function<void(const EventData&>> fn) {
        listeners_.push_back(fn);
    }
    // Fire event to all subscribers
    void notify(const EventData& data) const {
        for (auto& fn : listeners_) fn(data);
    }
};

// — Real-world use: stock price event system —————
struct PriceEvent { string symbol; double price; double change; };
struct OrderEvent { string symbol; int qty; bool isBuy; };

int main() {
    Observable<PriceEvent> priceStream;
    Observable<OrderEvent> orderStream;

    // UI logger — subscribes to price changes
    priceStream.subscribe([](const PriceEvent& e){
        cout << "[UI] " << e.symbol << " = $" << e.price
            << " (" << (e.change>=0?"+"+"") << e.change << ")\n";
    });

    // Alert system — subscribes to price changes
    priceStream.subscribe([](const PriceEvent& e){
        if (e.change < -5.0)
            cout << "[ALERT] " << e.symbol << " dropped by " << -e.change
                << "\n";
    });

    // Order logger
    orderStream.subscribe([](const OrderEvent& e){
        cout << "[ORDER] " << (e.isBuy?"BUY":"SELL") << " "
            << e.qty << " x " << e.symbol << "\n";
    });

    priceStream.notify({"AAPL", 175.50, +2.3});
    priceStream.notify({"TSLA", 180.20, -8.1}); // triggers alert
    orderStream.notify({"AAPL", 100, true});
}
```

SECTION 08

Best Practices & Summary

Tips · Pitfalls · Modern C++ · Key takeaways

BEST PRACTICES FOR TEMPLATE PROGRAMMING

✔ Keep definitions in headers

Template bodies must be visible at instantiation. Put them in .h/.hpp (or explicit instantiation in .cpp).

✔ Prefer typename over class

Use 'typename T' (not 'class T') unless T must itself be a class template — avoids confusion.

✔ Use static_assert for constraints

static_assert(is_arithmetic<T>::value, "T must be numeric") gives clear error at compile time.

⚠ Expect cryptic errors

Template error messages can be 100s of lines. Use C++20 concepts to add constraints and get cleaner errors.

✗ Don't rely on implicit conversions

Templates deduce types exactly — add<int>(3, 4.5) fails because int≠double. Use explicit cast.

✗ Don't forget const correctness

Generic code with const errors is hard to debug. Use const T& for parameters you don't modify.

✔ Use meaningful parameter names

Use T for type, N for size, K/V for key/value, Comp for comparator. Aids readability greatly.

✔ Default parameters for common cases

template<typename T, typename Comp=less<T>> lets users omit the comparator most of the time.

⚠ Watch for code bloat

Each instantiation generates separate code. Many distinct types → large binary. Use explicit instantiation to control.

✗ Don't over-templating

If a function only makes sense for one type, make it a regular function. Templates add compile complexity.

✗ Don't specialize function templates

Prefer overloads instead. Function template specialization interacts badly with overload resolution.

✗ Don't put template bodies in .cpp without explicit instantiation

Causes linker errors — the most common template beginner mistake.

COMMON TEMPLATE PITFALLS & FIXES

1. Template definition in .cpp (Linker Error)

✗ WRONG

```
// myStack.cpp
template<typename T>
void Stack<T>::push(T v) { /*...*/ }
// LINKER ERROR when used from
// main.cpp!
```

✓ FIX

```
// Put definition in header myStack.h
// OR add explicit instantiation:
template class Stack<int>; // in
.cpp
```

2. Ambiguous deduction — mixed types

✗ WRONG

```
template<typename T>
T add(T a, T b){return a+b;}
add(3, 4.5); // ERROR: int vs
double
```

✓ FIX

```
template<typename T1, typename T2>
auto add(T1 a, T2 b){return a+b;}
add(3, 4.5); // OK: T1=int, T2=double
```

3. Missing operator constraint

✗ WRONG

```
template<typename T>
T sum(vector<T>& v){
    T r{}; for(auto& x:v) r+=x;
    return r;
}
sum(boolVec); // ERROR if T has no
+=
```

✓ FIX

```
// C++20 concept
template<typename T>
requires requires(T a){a+=a;}
T sum(vector<T>& v){...}
```

4. Template specialization for function

✗ WRONG

```
template<> void print<bool>(bool
v){ /*...*/ }
// Specialization doesn't
participate
// in overload resolution –
confusing!
```

✓ FIX

```
// Prefer non-template overload:
void print(bool v){
    cout<<(v?"true":"false"); }
// Regular overload beats template
```

TEMPLATES VS POLYMORPHISM – WHEN TO USE WHAT

Criteria	Templates (Compile-time)	Virtual Functions (Runtime)
When types are known	At compile time	At runtime
Performance	Zero runtime overhead	Indirect call via vtable (small cost)
Best use case	Performance-critical code	Plugin / extensibility systems
Type list	Closed at compile time	Open to extension
Error messages	Complex (pre-C++20)	Simpler and clearer
Flexibility	Less flexible at runtime	Highly flexible at runtime

MODERN C++ TEMPLATE FEATURES

C++11 Variadic templates

```
template<typename... Args>
void print(Args... args){...}
```

C++11 Extern templates

```
extern template class vector<int>;
// Suppresses instantiation here
```

C++17 Class Template Deduction (CTAD)

```
auto p = pair(1, 3.14);
// deduced: pair<int,double>
```

C++20 Concepts

```
template<typename T>
concept Numeric = is_arithmetic_v<T>;
template<Numeric T> T add(T a,T b);
```

C++11 Alias templates

```
template<typename T>
using Vec = vector<T>;
Vec<int> v; // = vector<int>
```

C++14 Generic lambdas

```
auto add = [](auto a, auto b){
    return a+b;
}; // T deduced per call
```

C++17 if constexpr

```
if constexpr(is_integral_v<T>)
    return x%2==0; // compile-time if
```

C++20 Abbreviated templates

```
// Even cleaner syntax:
auto square(auto x){ return x*x; }
// equivalent to template<typename T>
```

GENERIC PROGRAMMING — QUICK REFERENCE CHEAT SHEET

CHEAT SHEET

```
// — Function template —————
template <typename T>
T square(T x) { return x*x; }
auto r = square(4);           // T deduced as int
auto r = square<double>(4);    // T explicitly double
```

```
// — Class template —————
template <typename T, int N=10>
class Ring { T data_[N]; int head_=0; ... };
Ring<int>      r1;           // T=int, N=10 (default)
Ring<string,5> r2;           // T=string, N=5
```

```
// — Multiple type params —————
template <typename K, typename V>
class Dict { map<K,V> data_; ... };
```

```
// — Full specialization —————
template <> class TypeInfo<int> { static string
name(){return "int";} };
```

```
// — Partial specialization —————
template <typename T> class Box<T,T> { T sum(){ return
first+second; } };
```

```
// — Variadic template —————
template <typename... Args>
auto sumAll(Args... args) { return (... + args); } //
C++17 fold
```

```
// — Non-type parameter —————
template <typename T, int SIZE>
class FixedArray { T data_[SIZE]; };
```

```
// — Concept (C++20) —————
template <typename T>
concept Addable = requires(T a, T b) { a + b; };
template <Addable T> T add(T a, T b) { return a+b; }
```

```
// — STL usage reminders —————
vector<int>  v = {3,1,4,1,5};
sort(v.begin(), v.end());
auto it = find_if(v.begin(), v.end(), [](int x){ return
x>3; });
int s = accumulate(v.begin(), v.end(), 0);
```

GENERIC PROGRAMMING IN C++

THANK YOU